

AI Assisted Coding-15.4

Name: G.Harika

Ht.no:2303A51612

Batch.no:05

Task 1:

Prompt:

Create a simple REST API for an Online Food Ordering System using Node.js and Express.

Include endpoints:

- GET /menu → list food items
- POST /order → create order
- GET /order/:id → get order status • PUT /order/:id → update order
- DELETE /order/:id → cancel order

Use in-memory storage, UUID for IDs, proper status codes, and return clean JSON responses. Provide complete working code in one file (server.js) with run instructions. Also suggest improvements like authentication and pagination.

Output :

localhost:3000/menu

pretty-print

```
{
  "id": 1,
  "name": "Pizza",
  "price": 250
},
{
  "id": 2,
  "name": "Burger",
  "price": 120
},
{
  "id": 3,
  "name": "Pasta",
  "price": 200
},
{
  "id": 4,
  "name": "Biryani",
  "price": 180
}
```

POST http://localhost:3000/order

Params Authorization Headers (9) Body Scripts Settings

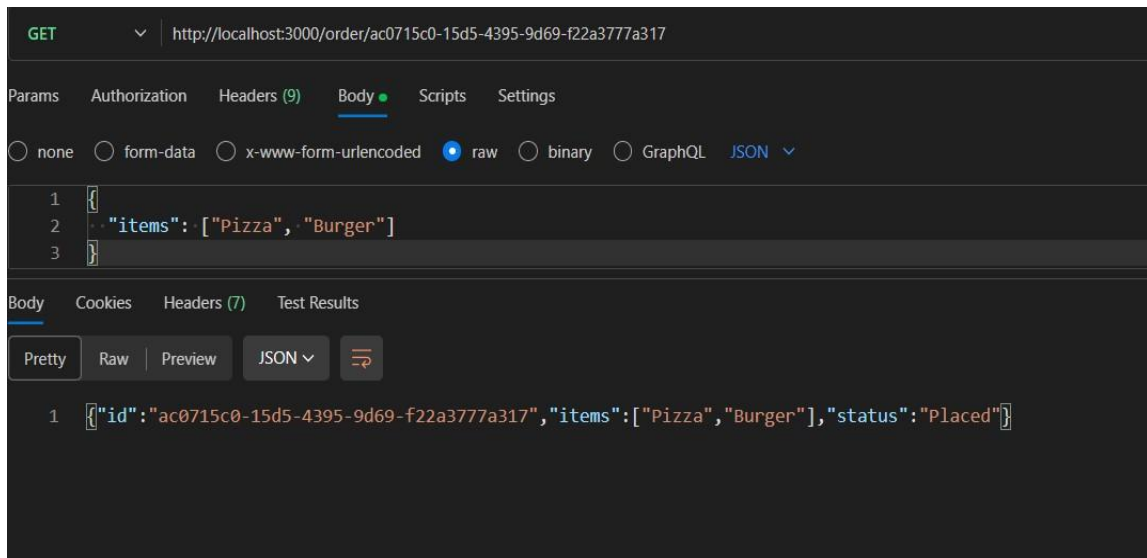
none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "items": ["Pizza", "Burger"]
3 }
```

Body Cookies Headers (7) Test Results

Pretty Raw Preview JSON

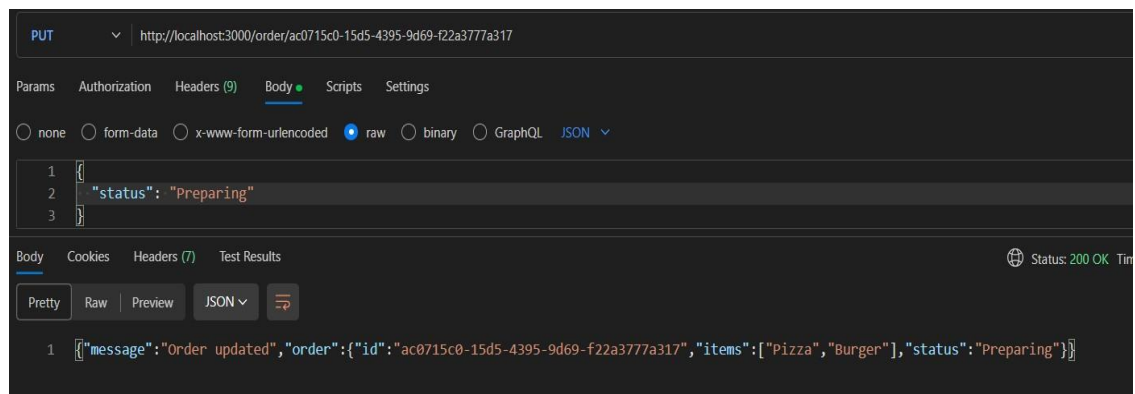
```
1 {
2   "message": "Order placed successfully",
3   "order": {
4     "id": "ac0715c0-15d5-4395-9d69-f22a3777a317",
5     "items": [
6       "Pizza",
7       "Burger"
8     ],
9     "status": "Placed"
10  }
```



```
GET http://localhost:3000/order/ac0715c0-15d5-4395-9d69-f22a3777a317

Params Authorization Headers (9) Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
1 {
2   "items": ["Pizza", "Burger"]
3 }

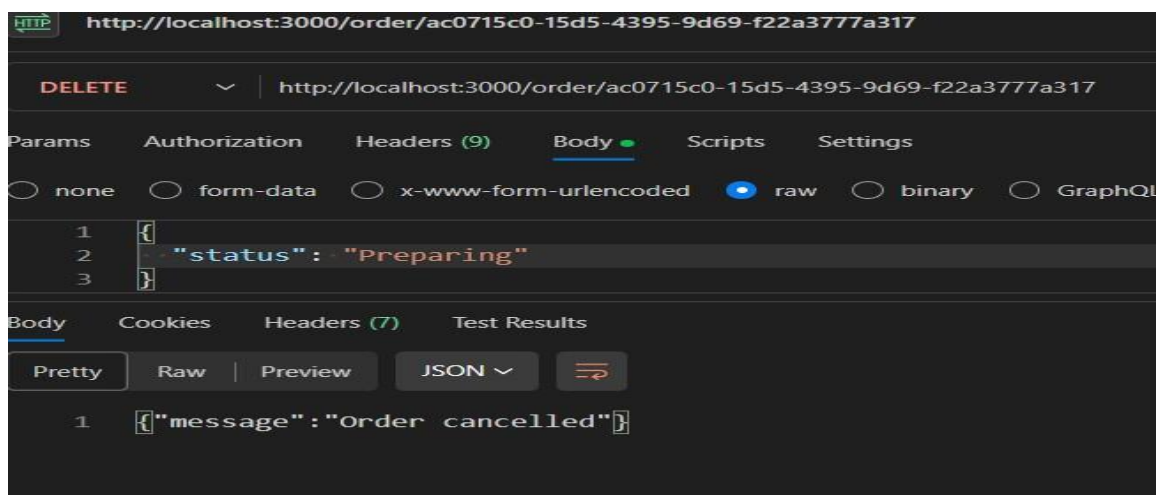
Body Cookies Headers (7) Test Results
Pretty Raw Preview JSON
1 [{"id": "ac0715c0-15d5-4395-9d69-f22a3777a317", "items": ["Pizza", "Burger"], "status": "Placed"}]
```



```
PUT http://localhost:3000/order/ac0715c0-15d5-4395-9d69-f22a3777a317

Params Authorization Headers (9) Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
1 {
2   "status": "Preparing"
3 }

Body Cookies Headers (7) Test Results Status: 200 OK Time
Pretty Raw Preview JSON
1 [{"message": "Order updated", "order": {"id": "ac0715c0-15d5-4395-9d69-f22a3777a317", "items": ["Pizza", "Burger"], "status": "Preparing"}}]
```



```
DELETE http://localhost:3000/order/ac0715c0-15d5-4395-9d69-f22a3777a317

Params Authorization Headers (9) Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL
1 {
2   "status": "Preparing"
3 }

Body Cookies Headers (7) Test Results
Pretty Raw Preview JSON
1 [{"message": "Order cancelled"}]
```

Explanation :

This code implements a simple API for an online food ordering system using Flask. It allows users to view the menu, place orders, track orders, update existing orders, and cancel orders. The menu and orders are stored in dictionaries, and the API handles various HTTP methods to perform the necessary operations on the orders.

Task 2

Prompt:

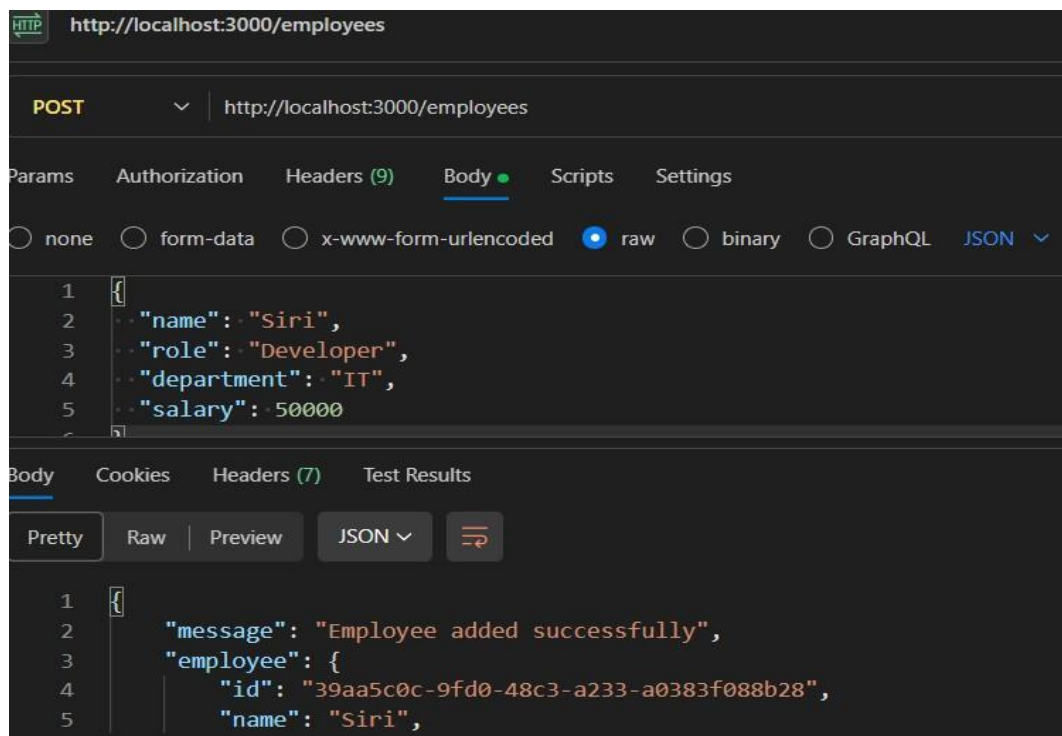
Create a REST API for an Employee Payroll System using Node.js and Express.

Include endpoints:

- GET /employees → list all employees
- POST /employees → add employee with salary details
- PUT /employees/:id/salary → update employee salary
- DELETE /employees/:id → remove employee

Use in-memory storage, unique IDs, input validation, and proper HTTP status codes. Suggest a clear data model structure (employee id, name, role, salary). Add comments/docstrings for each endpoint. Provide complete working code in one file and basic API documentation. Also ensure secure and structured data handling.

Output:



HTTP

http://localhost:3000/employees

GET

http://localhost:3000/employees

Params

Authorization

Headers (9)

Body

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

```
1 {
2   "name": "Siri",
3   "role": "Developer",
4   "department": "IT",
5   "salary": 50000
6 }
```

Body

Cookies

Headers (7)

Test Results

Pretty

Raw

Preview

JSON

```
1 [
2   {
3     "id": "39aa5c0c-9fd0-48c3-a233-a0383f088b28",
4     "name": "Siri",
5     "role": "Developer",
```

HTTP

http://localhost:3000/employees/39aa5c0c-9fd0-48c3-a233-a0383f088b28/salary

PUT

http://localhost:3000/employees/39aa5c0c-9fd0-48c3-a233-a0383f088b28/salary

Params

Authorization

Headers (9)

Body

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

```
1 {
2   "salary": 70000
3 }
```

Body

Cookies

Headers (7)

Test Results

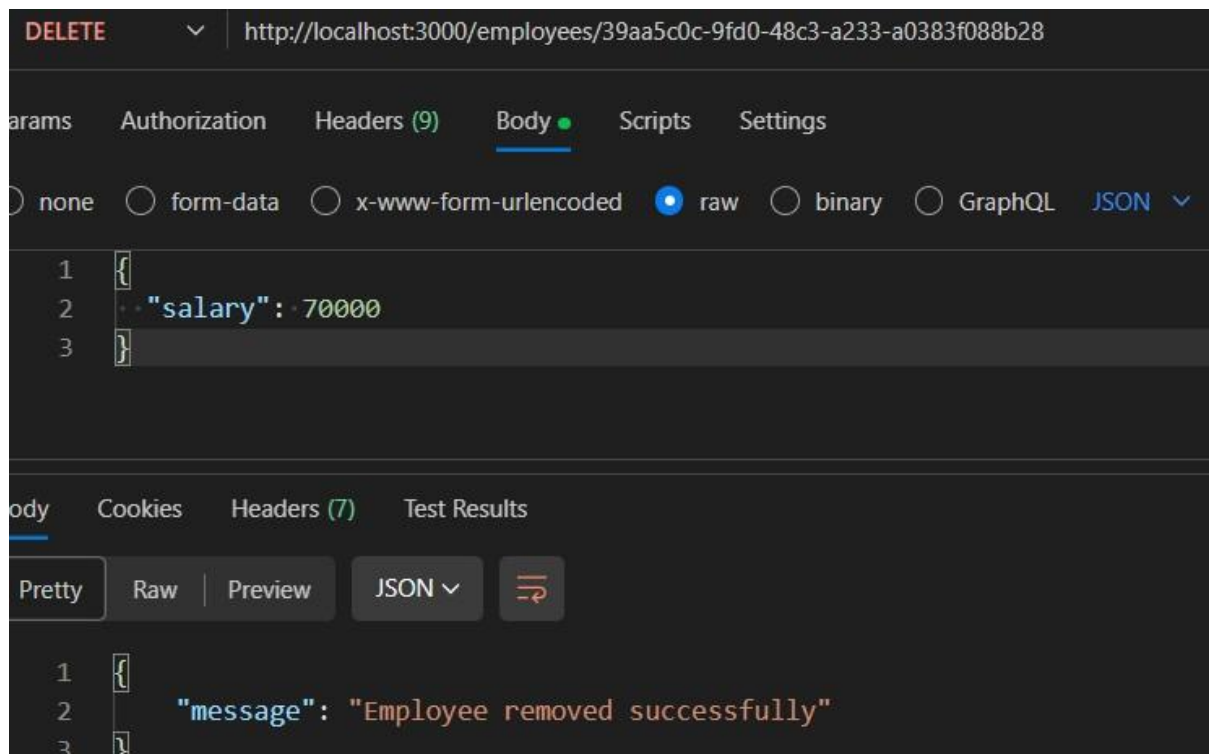
Pretty

Raw

Preview

JSON

```
1 {
2   "message": "Salary updated successfully",
3   "employee": {
4     "id": "39aa5c0c-9fd0-48c3-a233-a0383f088b28",
5     "name": "Siri",
```



Explanation :

The Employee Payroll API built using Flask manages employee data through REST endpoints. It allows users to list employees, add new employees, update employee salary, and delete employees using GET, POST, PUT, and DELETE methods. Employee information is stored in an in-memory list and returned in JSON format.

Task 3:

Prompt:

Create a Student Management REST API using Node.js and Express.

Features:

- Add student (name, rollNumber, course, year)
- Get all students
- Get student by ID
- Update student
- Delete student

Use:

- In-memory storage
- UUID for IDs
- Input validation - Proper status codes

Output:

- Complete server.js with comments **Output:**

The image displays three sequential screenshots of a REST client interface, likely Postman, showing the configuration and results of HTTP requests to a local server at `http://localhost:3000`.

First Screenshot (POST): The request is a `POST` to `http://localhost:3000/students`. The body is set to `raw` and contains the following JSON:

```
{
  "name": "Siri",
  "rollNumber": "22CS001",
  "course": "B.Tech CSE",
  "year": 3
}
```

The response is shown in `JSON` format:

```
{
  "message": "Student added successfully",
  "student": {
    "id": "0d8e4a78-d926-482e-8176-51a52bca7793",
    "name": "Siri",
    "rollNumber": "22CS001",
    "course": "B.Tech CSE",
    "year": 3
  }
}
```

Second Screenshot (PUT): The request is a `PUT` to `http://localhost:3000/students/0d8e4a78-d926-482e-8176-51a52bca7793`. The body is set to `raw` and contains the following JSON:

```
{
  "year": 4
}
```

The response is shown in `JSON` format:

```
{
  "message": "Student updated successfully",
  "student": {
    "id": "0d8e4a78-d926-482e-8176-51a52bca7793",
    "name": "Siri",
    "rollNumber": "22CS001",
    "course": "B.Tech CSE",
    "year": 4
  }
}
```

Third Screenshot (DELETE): The request is a `DELETE` to `http://localhost:3000/students/0d8e4a78-d926-482e-8176-51a52bca7793`. The body is set to `raw` and contains the following JSON:

```
{
  "year": 4
}
```

The response is shown in `JSON` format:

```
{
  "message": "Student deleted successfully"
}
```

127.0.0.1:5000/weather/forecast/<city

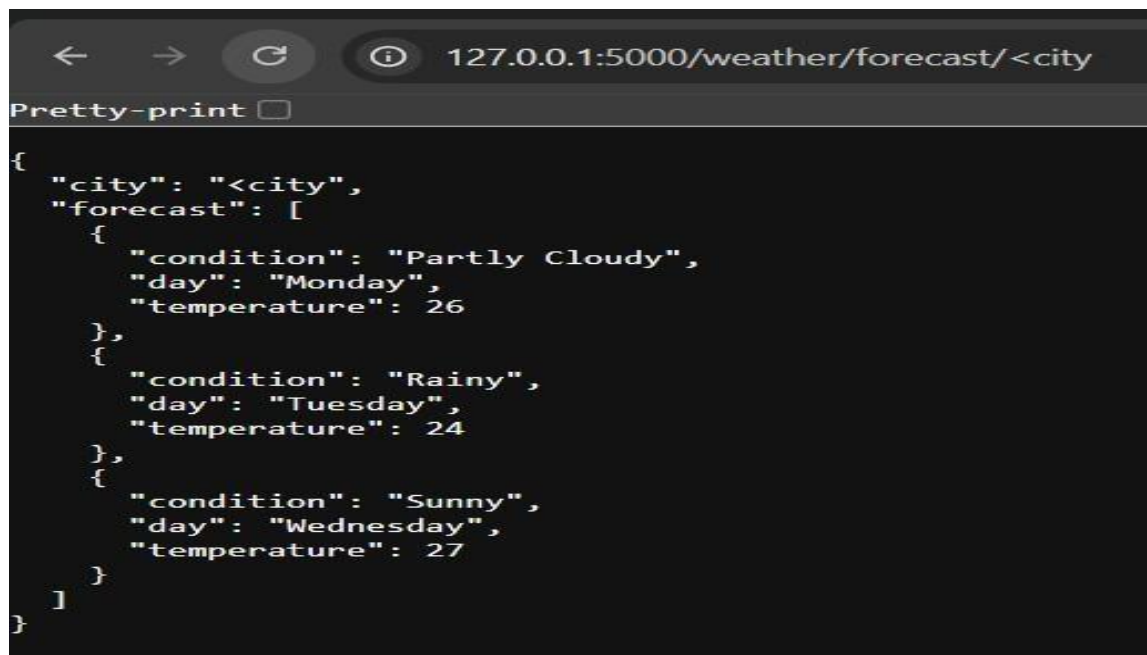
```
{
  "city": "<city",
  "forecast": [
    {
      "condition": "Partly Cloudy",
      "day": "Monday",
      "temperature": 26
    },
    {
      "condition": "Rainy",
      "day": "Tuesday",
      "temperature": 24
    },
    {
      "condition": "Sunny",
      "day": "Wednesday",
      "temperature": 27
    }
  ]
}
```

Use OpenWeatherMap API with axios.

Return temperature, humidity, and weather condition in JSON.

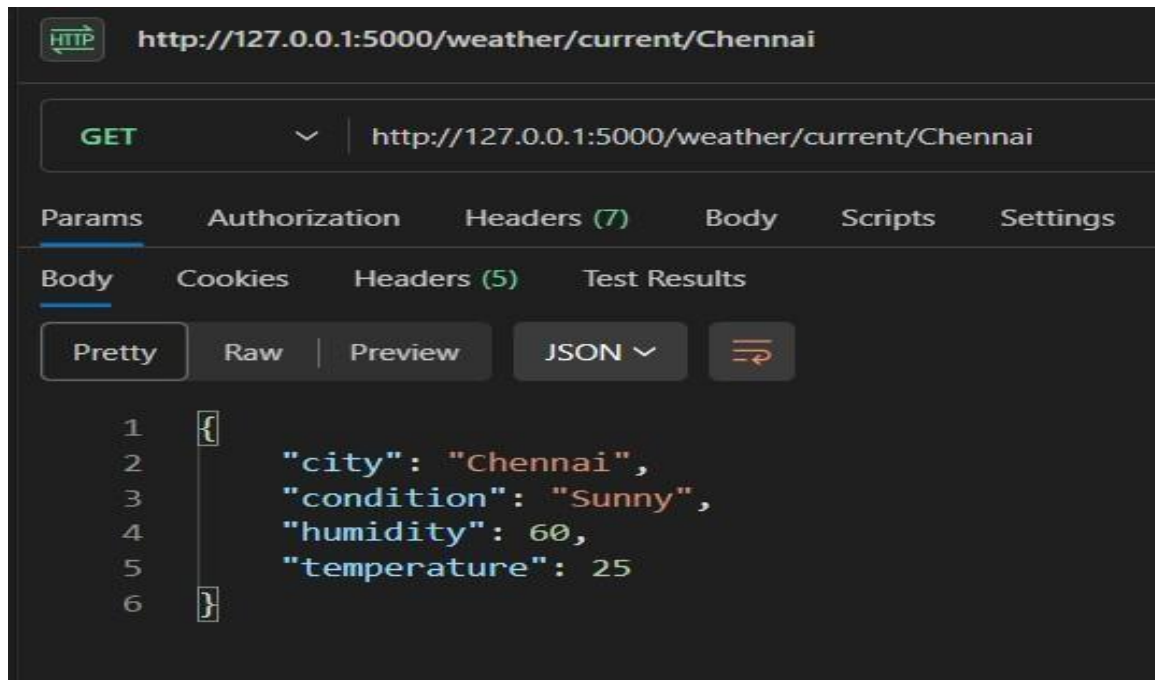
Include error handling and clean code.

Output:



A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:5000/weather/forecast/<city>`. Below the address bar, there is a "Pretty-print" checkbox. The main content area displays a JSON object representing a weather forecast. The JSON is formatted with syntax highlighting and line numbers.

```
{
  "city": "<city>",
  "forecast": [
    {
      "condition": "Partly Cloudy",
      "day": "Monday",
      "temperature": 26
    },
    {
      "condition": "Rainy",
      "day": "Tuesday",
      "temperature": 24
    },
    {
      "condition": "Sunny",
      "day": "Wednesday",
      "temperature": 27
    }
  ]
}
```



A screenshot of a web browser window showing a REST client interface. The URL bar displays `http://127.0.0.1:5000/weather/current/Chennai`. The interface includes tabs for "Params", "Authorization", "Headers (7)", "Body", "Scripts", and "Settings". The "Body" tab is selected, and within it, the "Pretty" view is chosen. The JSON response is displayed with syntax highlighting and line numbers.

```
1 {
2   "city": "Chennai",
3   "condition": "Sunny",
4   "humidity": 60,
5   "temperature": 25
6 }
```

Explanation :

This code creates a simple Flask REST API with two endpoints: `/weather/current/<city>` and `/weather/forecast/<city>`. The first endpoint returns mock current weather data for the specified city, while the second endpoint returns a mock weather forecast for the next few

days. In a real implementation, you would replace the mock data with actual data fetched from the OpenWeatherMap API.

TASK – 05

Prompt :

Create a Python **Flask REST API endpoint to delete an item from a list using the DELETE method. The endpoint should be /items/<int:index>. If the index is invalid, return a JSON error message "Item not found" with status code 404. If successful, remove the item and return a JSON response with "message": "Item deleted" and the deleted item.

Output:

HTTP **http://127.0.0.1:5000/items/0**

DELETE **http://127.0.0.1:5000/items/0**

Params Authorization Headers (7) Body Scripts

Query Params

	Key
	Key

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
1 {  
2   "item": "item1",  
3   "message": "Item deleted"  
4 }
```

HTTP **http://127.0.0.1:5000/items/5**

DELETE **http://127.0.0.1:5000/items/5**

Params Authorization Headers (7) Body Scripts Se

Query Params

	Key
	Key

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
1 {  
2   "error": "Item not found"  
3 }
```

Explanation :

This code creates a simple Flask REST API with an endpoint to delete an item from a list. The endpoint is defined as `/items/<int:index>`, where `<int:index>` is a placeholder for the index of the item to be deleted.

When a DELETE request is made to this endpoint, the function `delete_item` is called with the index as an argument. The function checks if the index is valid (i.e., within the bounds of the list). If it is valid, it removes the item from the list using `pop()` and returns a JSON response indicating that the item was deleted, along with the deleted item itself.